Title: Learned token-per-ball simulator for bouncing-ball physics, with baselines, ablations and a scaling study

Project type: Applied project.

Project idea. I will build, evaluate and demonstrate a learned simulator for 2D bouncing balls (walls, ball-ball contact, gravity) in which each ball is one state token (position, velocity) and a small network predicts the next state. Learned simulators matter because exact solvers get costly at scale, but learned ones often drift in energy and fail on long rollouts. The question: does a physics-structured token model beat a generic graph network in accuracy, energy behavior and generalization to far more balls than it was trained on? Prior work: before the course I built a prototype (exact simulator, a trained model, and a browser port). The semester work is the evaluation, baselines, ablations, scaling study and demo below.

Related work. Graph Network-based Simulators (Sanchez-Gonzalez et al., ICML 2020, with DeepMind's reference code), Interaction Networks (Battaglia et al., NIPS 2016), MeshGraphNets (Pfaff et al., ICLR 2021), Neural Relational Inference (Kipf et al., ICML 2018), Hamiltonian Neural Networks (Greydanus et al., NeurIPS 2019), Lagrangian Neural Networks (Cranmer et al., 2020). Code reference: the GNS implementation in github.com/google-deepmind/deepmind-research (learning_to_simulate), used for design only; my baseline is written from scratch in PyTorch.

Methods and pipeline. Data: a seeded GPU force simulator (walls, contact, gravity) generates trajectories of 1 to 100 balls in boxes of several sizes (about 1,200 training and 100 validation scenes of 24 steps). Model (existing, no attention): a pair-force network and a wall-force network, both functions of distance only, plus a learned gravity, integrated with eight velocity-Verlet substeps over a linear-time neighbor graph (about 8.7k weights). Prior result to build on: trained on at most 100 balls, it stays finite on a 50,000-ball test, with mean position error 0.28 at tick 10 versus the exact simulator. Training: unrolled multi-step loss on position and velocity (1, 4, then 8 steps), 50 epochs, under one hour on a single GPU. Baselines: constant velocity, an older attention-plus-GRU token model, and a new message-passing graph network (GNS style) trained on the same data. Evaluation: mean position error per ball at ticks 1 to 20, energy and momentum drift, wall restitution versus speed, and stability at 100, 1,000 and 50,000 balls, on three held-out seed sets with confidence intervals. Beyond tick 20 rollouts are chaotic, so I judge accuracy there by physical-law metrics. Ablations: substep count, an attention residual, fixed versus fine-tuned gravity, unroll length.

Planned implementation (15 weeks). Weeks 1-2: proposal approval, one-command pipeline (generate, train, evaluate, export) tested on a clean clone. Weeks 3-5: graph-network baseline with tests. Weeks 6-7: evaluation harness and results table (milestone: baseline comparison). Weeks 8-10: ablations and gravity fine-tune. Weeks 11-12: scaling study (box size, ball count, speed beyond training). Weeks 13-14: demo polish, failure analysis, report. Week 15: final presentation. Training uses one GPU on a university cluster. Pass/kill rule set in advance: the structured model must beat the graph network at tick 10 and in energy drift on all seed sets, otherwise I report the negative result and the trade-offs. Risk fallback: drop the scaling study first.

Planned demo. The live browser page (already built, faraday-dot-py.github.io/Augur/web/token/) running the model next to the exact simulator with energy plots and editable weights, a side-by-side truth, model and baseline video, and a results table with ablations.